

COSC 81/281: Problem Set 1: Search, Values, and Sampling

Due Tuesday, September 29 (11:59pm ET)

Problem sets are graded **pass/fail on completion**, and there are no late days. The problem sets are where the learning happens, and they are the best available practice for the midterm and final exam. Submit to **Gradescope** (<https://www.gradescope.com/courses/1372528>) and assign each question to a page in your submission. Questions are welcome on **Slack**. Starred (★) questions are required for COSC 281 students and optional (ungraded) for COSC 81 students.

AI Policy. You may use any AI tools you like to assist you with your assignments, and you should, as that is likely how you will work for the rest of your career. That being said, the written portions are practice for the in-class midterm and exam, which are closed book and on paper. You should strive to answer these yourself and use AI as a tutor and not an oracle.

Please include an AI Tool Disclosure Below

Overview

This problem set examines three approaches to robot motion planning: search, value-based methods, and sampling. You will implement A* to find shortest paths on a grid, value iteration and policy iteration to compute policies for a grid-based Markov decision process (MDP), and a Rapidly-exploring Random Tree (RRT) to plan motions for a car with a minimum turning radius. By comparing these methods on the same maps, you will examine how the state representation and computational cost affect the resulting plans. The written problems provide practice with the calculations, proofs, and reasoning expected on exams.

Getting the code. Pull the course repository. The starter code is in `ps1/code/`, and the three maps, `open`, `maze`, and `narrow`, are in `sim/maps/`. The starter provides the search frontier, grid MDP, car motion primitives, collision checks, and plotting functions. Your implementations belong in `astar.py`, `vi.py`, `policy_iteration.py`, and `rrt.py`.

A* tie-breaking convention. Expand the frontier node with the smallest $f = g + h$. If multiple nodes have the same f -value, choose the node with the smallest h . If these values are also equal, choose the smallest row index, followed by the smallest column index. Row and column indices begin at 0 in the top-left corner. Use this convention for both the written problems and the programming task; the provided `DeterministicFrontier` implements it.

What to submit. Submit the following to **Gradescope** (<https://www.gradescope.com/courses/1372528>):

1. **Written work (PDF):** Include your answers to Part 1 and the tables, figures, and explanations requested in Part 2. You may write in the answer boxes, annotate the PDF, or prepare a separate document. To use `ps1/handout.tex`, enter your responses below the `%%%% SOLUTION HERE %%%%` markers in the corresponding `studentanswer` environments and compile the file. The boxes expand to accommodate typed responses. You may add pages for figures or longer answers. Assign each question to its corresponding page or pages when uploading.
2. **Code:** Submit `astar.py`, `vi.py`, `policy_iteration.py`, and `rrt.py`. Before submitting, run `python ps1/code/autograder.py` from the repository root.

Part 1: Written problems

These problems provide practice for the types of questions asked on exams. Show your work. Numerical answers can be expressed as small integers or simple fractions, so a calculator is not required.

P1: A*

The grid below is 4-connected with unit step cost. Dark cells are obstacles. You start at $S = (0, 0)$ with goal $G = (3, 3)$. Coordinates are given as (row, column), with S in the top-left corner. Use Manhattan distance to G as the heuristic; its value is printed in each open cell other than the goal, where it is zero. Break ties in f using the smaller heuristic value, then the smaller row index, and finally the smaller column index.

	0	1	2	3
0	S 6	5	4	3
1	5			2
2	4	3	2	1
3	3	2	1	G

- (a) List the first **five** nodes that A* expands, in order, and give the f -value of each node at expansion. For each tie, identify the rule that determines which node is expanded.

Answer / work

- (b) State the cost of the path returned by A* and write out one optimal path.

Answer / work

- (c) The node (1,0) is on the frontier with $f = 6$ after the first expansion, but A* reaches the goal without expanding it. Explain in two sentences how the tie-breaking rules produce this result.

Answer / work

P2: Heuristic admissibility

Consider the following heuristics for a cell n and goal G on a 4-connected unit-cost grid with obstacles. Let $\Delta r = |r_n - r_G|$ and $\Delta c = |c_n - c_G|$:

$$h_A(n) = \max(\Delta r, \Delta c), \quad h_B(n) = \Delta r + \Delta c + 2N_{\text{obs}},$$

where N_{obs} counts the obstacle cells in the axis-aligned rectangle whose opposite corners are n and G , including its boundary. The additional term in h_B is intended to account for detours around obstacles. In the grid below, $G = (0, 2)$ and the only obstacle is at $(1, 1)$.

	0	1	2
0			G
1			
2			

- (a) State the definition of an admissible heuristic.

Answer / work

- (b) Prove in two or three sentences that h_A is admissible on every 4-connected unit-cost grid, including grids with obstacles.

Answer / work

- (c) Determine whether h_B is admissible. If it is, provide a proof. If it is not, identify a cell in the grid above, compute both h_B and the optimal cost-to-go at that cell, and explain why counting obstacles does not give a valid lower bound.

Answer / work

P3: Value iteration

The 4×4 grid below uses (row, column) coordinates, with row 0 at the top and column 0 at the left. Dark cells are walls. Actions move deterministically north, south, east, or west. An action that would cross a wall or grid boundary leaves the robot in its current cell. Entering the goal $G = (0, 3)$ gives reward $+4$ and ends the episode; all other transitions give reward 0. Set $V(G) = 0$, use discount factor $\gamma = \frac{1}{2}$, and initialize $V_0 = 0$ at every cell. A synchronous sweep computes all entries of V_{k+1} using only V_k .

r/c	0	1	2	3
0				G
1				
2				
3		S		

- (a) Perform one synchronous sweep. Use the grid provided to enter V_1 at every free cell.

Answer / work

	0	1	2	3
0				G
1				
2				
3		S		

- (b) Perform a second synchronous sweep. Use the grid provided to enter V_2 at every free cell.

Answer / work

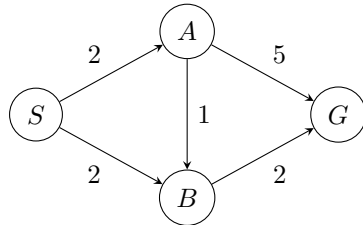
	0	1	2	3
0				G
1				
2				
3		S		

- (c) ★ (required for 281; optional for 81) For the start $S = (3, 1)$, identify the first sweep at which $V_k(S) \neq 0$ and compute its value. Explain how the shortest-path distance allows you to answer without performing each intermediate sweep.

Answer / work

P4: Admissibility and consistency on a graph

Consider the directed graph below, with start S , goal G , and positive edge costs. Only the arrows shown are available. For every heuristic, assume $h(G) = 0$ and nonnegative values. Admissibility requires $h(n) \leq h^*(n)$ at every node, where $h^*(n)$ is the least cost to the goal. Consistency additionally requires $h(u) \leq c(u, v) + h(v)$ for every directed edge $u \rightarrow v$.



	S	A	B	G
h_1	4	3	2	0
h_2	4	3	0	0
h_3	5	3	2	0

- (a) Compute h^* at all four nodes. Classify each listed heuristic as admissible or inadmissible and as consistent or inconsistent. For every failed property, give a node or edge that demonstrates the failure.

Answer / work

- (b) Keep $h(S) = 4$, $h(A) = 3$, and $h(G) = 0$, but set $h(B) = x$. Determine all $x \geq 0$ for which the heuristic is admissible, and all $x \geq 0$ for which it is consistent. Show the relevant bounds.

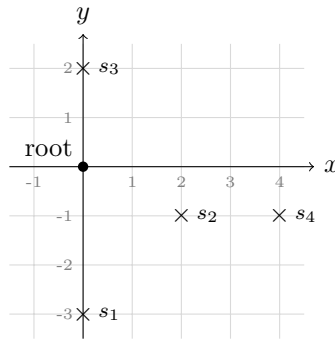
Answer / work

- (c) Restore h_1 from the table. Consider two independent changes to the original graph: change the cost of $A \rightarrow G$ from 5 to 2, or change it from 5 to 7. For each change, determine whether h_1 remains admissible and consistent. Justify your conclusions.

Answer / work

P5: RRT construction

An RRT initially contains only the root $(0, 0)$ in an obstacle-free plane. At each iteration, select the existing tree node with the smallest Euclidean distance to the sample, then add a node exactly one unit from that node in the direction of the sample. Process the samples in the order $s_1 = (0, -3)$, $s_2 = (2, -1)$, $s_3 = (0, 2)$, and $s_4 = (4, -1)$, as labeled in the figure.



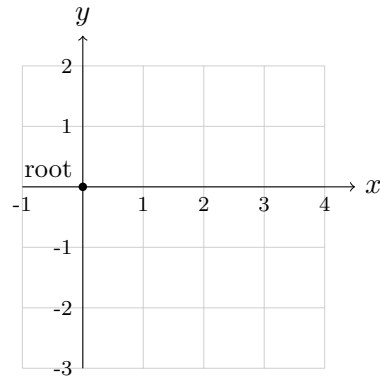
- (a) For each iteration, give the nearest existing node and the new node.

Answer / work

P5 (continued)

- (b) Draw the final tree on the coordinate grid, label the new nodes, and indicate the nearest neighbor selected at iteration 2. If using LaTeX, add a line inside the provided `tikzpicture` with `\draw (x1,y1) - (x2,y2);`, replacing the endpoint coordinates with those of the edge.

Answer / work



- (c) Explain why the nearest node at iteration 2 is not the root. Support your answer with the relevant distances.

Answer / work

- (c) Prove that h_k is consistent. Consider the cases $h^*(n) \leq k$ and $h^*(n) > k$ separately.

Answer / work

- (d) Using parts (b) and (c), determine whether A* tree search and A* graph search with a closed set and no reopening return optimal paths under h_k . Identify the sufficient property for each guarantee.

Answer / work

- (e) If $k \geq \max_n h^*(n)$, what algorithm does A* become? State whether it remains optimal and explain the effect on search effort.

Answer / work

Part 2: Programming

In this part, you will complete selected steps of three planning methods and compare their behavior on the supplied maps. These instructions are self-contained: you do not need results from Part 1 to implement or run the code. The provided programs load the maps, call your functions, measure their behavior, and generate the plots. Your responsibility is to complete the labeled TODO blocks and interpret the resulting output.

Before you begin. Use the course Python environment from PS0. Pull the course repository to obtain this assignment, activate that environment, and open a terminal in the repository root, the folder containing `sim/`, `ps0/`, and `ps1/`. All commands in this section are run from that folder. If you need to recreate the environment, follow the installation instructions in the repository's root `README.md`. These experiments save images to files; they do not open an interactive driving window.

What to edit. Your submission consists of four files in `ps1/code/`:

- `astar.py`: complete TODO A, which updates the search when a cheaper route to a neighboring cell is found.
- `vi.py`: complete TODO V, the value update for one state, and TODO P, the greedy action choice for one state.
- `policy_iteration.py`: complete TODO I by reusing your greedy action-selection function. The remaining policy-iteration code is provided.
- `rrt.py`: complete TODO N, nearest-node selection, and TODO E, a call to the provided steering function.

Replace each corresponding `raise NotImplementedError(...)` with your implementation. Keep the function names, arguments, and return formats unchanged. The surrounding scaffold is provided; you are not expected to rewrite it. Read its comments to understand how your result is used. Files described as provided are not part of your submission and will be supplied by the grader.

How the programs fit together. The following paths are relative to `ps1/code/`. A runner is the program you execute; it imports and calls the functions you complete.

Runner	Functions it calls	Provided support
<code>run_astar.py</code>	<code>astar</code> in <code>astar.py</code>	Map loading, <code>search_utils.py</code> , result plots.
<code>compare.py</code>	<code>value_iteration</code> , <code>extract_policy</code> , <code>policy_iteration</code> , and <code>astar</code>	<code>grid_mdp.py</code> , fixed-policy evaluation, timing and plots.
<code>run_rrt.py</code>	<code>rrt</code> in <code>rrt.py</code>	<code>dubins.py</code> , sampling, car motions, collision checks and plots.

Sections labeled *Provided scaffold and interface* explain supplied code; they are not implementation requirements. Only the TODOs require edits.

Suggested workflow and submission

Work through Tasks 1, 2, and 3 in order. For each task, complete its TODOs, run its public tests, and then run its experiment command. Task 2's comparison also calls your A* implementation, so complete Task 1 first. Task 3 does not depend on either of the other implementations.

The commands below run only the tests for the task you are working on:

```
# Task 1
python -m pytest ps1/code/tests/test_astar.py -q
# Task 2
python -m pytest ps1/code/tests/test_vi.py -q
python -m pytest ps1/code/tests/test_policy_iteration.py -q
# Task 3
python -m pytest ps1/code/tests/test_rrt.py -q
```

An unfinished TODO raises `NotImplementedError`; this is expected before you implement it. After completing a task, its tests should pass. If a test fails, read the function name and assertion in the failure report, then compare your implementation with the task's input and output contract. Public tests cover small cases; passing them does not replace running the supplied map experiments. Staff tests also exercise additional cases. Do not change the tests to make an implementation pass.

Once all three tasks are complete, run the full public suite:

```
python ps1/code/autograder.py
```

The complete coding report. Submit only two figures (the A* maze overlay and the RRT narrow plot), one three-row A*/VI/PI comparison table for maze, and the three short responses requested below, one per task. No other tables or plots are required. Run all three maps to check your implementations; the extra images and arrays are for your own inspection.

The experiment commands in the following pages save all requested images in `ps1/figures/`. Open those files to inspect your results. Fill the supplied comparison table in your PDF, or insert a legible screenshot of the three maze rows. Use your own measured results, including timing measurements from your machine. The runners already produce the figures; you do not need to write plotting code.

What to submit. Upload `astar.py`, `vi.py`, `policy_iteration.py`, and `rrt.py` as the code submission. Include the tables, images, and short explanations requested after each task in your written-work PDF. Do not paste your Python implementations into that PDF. Do not submit the simulator, helper files, tests, or saved `.numpy` arrays. If you use the editable TeX handout, insert each figure and explanation in its corresponding `studentanswer` environment. For example:

```
\includegraphics[width=0.9\linewidth]
  {ps1/figures/ps1_astar_maze.png}
```

Task 1: A* on occupancy grids

Task checklist. Open `ps1/code/astar.py` and complete TODO A. Run `test_astar.py` using the command in the workflow section, then run `run_astar.py` as shown below. Submit the maze overlay and a one- or two-sentence comparison of open and maze.

Goal and inputs. A* finds a minimum-cost path from a specified start cell to a goal cell. The runner loads each map, accounts for the robot's footprint, and calls `astar(occ, start, goal, heuristic=manhattan)`. You receive `occ`, a two-dimensional Boolean NumPy array, and two tuples `start` and `goal`. A tuple is `(row, column)`; row zero is at the top and column zero is at the left. `occ[row, column]` is `True` for an obstacle and `False` for a free cell. The robot moves north, south, east, or west, with cost 1 per move. There are no diagonal moves. You do not need to convert world coordinates or load an image inside your function.

A* orders discovered cells by $f(n) = g(n) + h(n)$, where $g(n)$ is the best cost found so far from the start and $h(n)$ estimates the remaining cost to the goal. The supplied default is Manhattan distance, $h(n) = |r_n - r_g| + |c_n - c_g|$, in grid moves. Always call the `heuristic(cell, goal)` argument rather than hard-coding this formula; the tests also use other heuristic functions.

Provided scaffold and interface. `search_utils.py` provides `DeterministicFrontier`, `manhattan`, and the `SearchResult` return type. `frontier.push(f, h, cell)` adds an entry; `frontier.pop()` removes the smallest entry and returns `(f, h, cell)`. The frontier orders entries by $(f, h, \text{row}, \text{column})$, with smaller values first. Thus you do not implement a priority queue or tie-breaking rule.

The starter initializes `g` (best discovered costs), `parent` (predecessors), `closed` (expanded cells), and `expanded` (expansion order). It also supplies the search loop, filters invalid neighbors, skips obsolete frontier entries, tests for the goal, and reconstructs the final path. Duplicate frontier entries are allowed: when you find a cheaper route, push a new entry and let the provided stale-entry check discard the old one later.

Your implementation: TODO A. The TODO runs once for each valid neighboring cell `nb` of the currently expanded `cell`. Update its route cost and A* priority:

1. Compute the candidate cost of reaching `nb` through `cell`. Add the unit edge cost to `g[cell]`.
2. Compare that cost with `g.get(nb, math.inf)`. A missing dictionary entry represents an undiscovered cell with infinite cost.
3. Only if the candidate is strictly smaller, update `g[nb]`, record `cell` as `parent[nb]`, and compute $h = \text{heuristic}(nb, goal)$. Push `nb` with priority $f = g[nb] + h$ (cost so far plus estimated remaining cost), passing h separately for tie-breaking. Otherwise, leave the search state unchanged.

These updates determine which cell the next frontier pop selects and which predecessor the provided path-recovery code follows.

Task 1: checking and running your implementation

Output contract. The scaffold returns `SearchResult(path, cost, expanded)`. The path must include both the start and the goal, and its cost is the number of moves. The `expanded` list records cells in expansion order, including the start and goal. An expansion occurs when a non-stale entry is removed from the frontier. Skip entries superseded by a cheaper route or belonging to an already expanded cell. If no path exists, return an empty path, infinite cost, and the expansion sequence accumulated before the frontier became empty. Test for the goal when removing a valid entry from the frontier.

The autograder checks expansion order as well as path cost. The provided scaffold enforces the expansion conventions; leave that code unchanged. For a small sanity check, two adjacent free cells have path cost 1, and the returned path includes both cells. If start and goal coincide, the cost is zero and the path contains that one cell.

Run your implementation on the three maps:

```
python ps1/code/run_astar.py --out-dir ps1/figures
```

The command prints the path cost in grid moves and the number of expanded nodes for each map. Each grid move corresponds to 0.05 m. It also saves a plot for each map showing the expanded cells and the returned path. The files are `ps1/figures/ps1_astar_open.png`, `ps1_astar_maze.png`, and `ps1_astar_narrow.png` in that same folder. No interactive window is expected. Inspect all three, but include only the maze overlay in your PDF; no A* table is required.

Code submission. Submit your implementation in `astar.py`.

PDF responses.

- (a) Include the A* overlay for **maze**.

Answer / work

- (b) In one or two sentences, report the expansion counts on **open** and **maze** and explain how obstacles affect the usefulness of the Manhattan heuristic. No separate table is needed.

Answer / work

Task 2: Value iteration and policy iteration

Task checklist. Complete TODOs V and P in `vi.py`, then TODO I in `policy_iteration.py`. Run `test_vi.py` and `test_policy_iteration.py`, then `compare.py`. Task 1 must work before this comparison can run. Submit only the maze comparison table and a two- or three-sentence interpretation; no plot is required.

Goal and model. Compute a value and a preferred action for every free cell using value iteration (VI) and policy iteration (PI). A value is the predicted discounted return from a cell; a policy specifies the action to take there. The runner constructs a `GridMDP` using cells of width 1.0 m. Both methods use the known transition and reward model; neither learns from sampled experience. Actions are north, south, east, and west. An attempted move into a wall or beyond the boundary leaves the robot in place. Entering the goal gives reward +10 and ends the episode; other transitions give zero. The terminal value is zero and $\gamma = 0.9$. Your functions must use their supplied `gamma` argument rather than hard-coding 0.9.

The world interface. Your functions receive a `world` object. You need only these members; map loading and transitions are already implemented:

- `world.shape` is (number of rows, number of columns).
- `world.free_cells()` returns the free-cell tuples, including the goal.
- `world.goal` is the terminal (row, column) tuple.
- `world.step(s, a)` returns (`s2`, `reward`, `done`): the successor cell, immediate reward, and a Boolean indicating termination. Actions are integer indices: 0 is north, 1 south, 2 east, and 3 west. Pass an index, not a direction vector.

A value array `V` has shape `world.shape`; `V[s]` accesses the value at tuple `s`. Walls contain `NaN`, and the goal contains zero. A policy array has the same shape but contains integer action indices; walls and the goal contain `-1`.

Your implementation: TODO V in `vi.py`. The scaffold supplies initialization, the sweep and state loops, and the convergence test for `value_iteration(world, gamma, tol, max_sweeps)`. For each free nonterminal state `s`, compute the four one-step returns

$$b_a = \begin{cases} r, & \text{if } \text{world.step}(s, a) \text{ reports termination,} \\ r + \gamma V_k(s'), & \text{otherwise,} \end{cases} \quad V_{k+1}(s) = \max_{a \in \{0,1,2,3\}} b_a.$$

Call `world.step` to obtain `s'` and `r` for each action. Write the maximum into `V_new[s]`. Read successor values from `V`, the previous sweep, not `V_new`; this is synchronous value iteration. The scaffold initializes all free cells to zero, retains zero at the goal, and leaves walls as `NaN`. It stops after the first sweep whose largest absolute value change is below `tol`, or after `max_sweeps`. It returns `(V, sweeps_run)`, counting the final sweep. You do not implement the stopping or counting logic.

Task 2: extracting and improving a policy

Your implementation: TODO P in `vi.py`. `extract_policy(world, V, gamma)` receives a value table and returns a greedy action grid. For each free nonterminal state, evaluate the same four one-step returns as in TODO V. Store the *index of the maximizing action* in `policy[s]`, not the maximum value. Evaluate actions in order 0, 1, 2, 3. In a tie, select the smaller action index; `np.argmax` on an action-ordered list gives this rule. Do not change the supplied `V`. Initialization and the state loop are provided, including the `-1` entries for walls and the goal.

Your implementation: TODO I in `policy_iteration.py`. The supplied PI loop starts with an all-north policy on nonterminal states. It alternates evaluating that fixed policy and improving it, stopping when the action grid no longer changes. The provided `evaluate_policy` function solves the fixed-policy equations; you do not need to implement or modify its linear algebra.

Your `improve_policy(world, V, gamma)` receives the values of the *current* policy and should return actions greedy with respect to that table. This is exactly the operation in `extract_policy`. Reuse that function: a one-line call returning its result is sufficient and intended. Do not run value iteration here, since that would replace evaluation of the current policy with a different computation.

How the runner uses your functions. `compare.py` constructs the world, calls your `value_iteration`, and passes its returned table to your `extract_policy`. It then calls the provided `policy_iteration` loop, which repeatedly calls `evaluate_policy` and your `improve_policy`. Finally, it calls your A* function for the fine-grid comparison. Complete Task 1 before running this full comparison.

Check and run. Use the two Task 2 test commands listed at the start of Part 2. As a sanity check, an action entering the goal has one-step return 10, not $10 + \gamma \cdot 10$: the goal has no future return. Then run:

```
python ps1/code/compare.py --plot-dir ps1/figures
```

The runner reports path length and planning time for each method, together with A* expansion counts, VI sweep counts, and PI evaluation/improvement rounds. These counts measure different operations: a PI round includes a policy-evaluation solve. Compare measured times as well as counts; do not infer a speedup from the number of rounds alone. The runner checks agreement between the two MDP value tables and saves `ps1/figures/ps1_values.png` and `ps1/code/out/values_{open,maze,narrow}.npy`. The arrays and value plot are for inspection, not submission. A failed value-table comparison is an error to investigate, not a result to include in your table. A* uses 0.05 m cells and VI/PI use 1.0 m cells; compare distances in meters, not raw numbers of steps.

Code submission. Submit `vi.py` and `policy_iteration.py`. The provided helper and loop should remain unchanged.

What the comparison can tell you. VI and PI use the same states, transitions, reward, and goal, so their values should agree. Their timing comparison concerns these two implementations on your machine. A* uses a finer grid and answers a single start-to-goal query rather than computing a policy for all states; its timing is contextual, not a controlled test of which algorithm is universally faster. RRT also imposes car-motion constraints and uses a different maze endpoint, so it is not part of this timing comparison.

PDF responses.

- (a) Fill the table below using only the three `maze` rows printed by `compare.py`. Keep the work-count units: expansions for A*, sweeps for VI, and rounds for PI. You may fill the supplied table or replace it with a legible screenshot of those three rows.

Answer / work				
	Method	Path (m)	Work count (with unit)	Time (ms)
	A*			
	VI			
	PI			

- (b) In two or three sentences, identify whether VI or PI was faster on your machine and explain why comparing sweep and round counts alone would not establish that result. Give one reason the A* timing is not a like-for-like comparison with these two methods.

Answer / work

Task 3: RRT with car motion constraints

Task checklist. Open `ps1/code/rrt.py` and complete TODOs N and E. Run `test_rrt.py`, then `run_rrt.py`. Submit the narrow plot and a two- or three-sentence explanation of the least efficient map's sample-to-node ratio. No results table is required.

Goal and representation. A rapidly exploring random tree (RRT) grows a tree of feasible car motions from the start toward sampled states. Unlike the grid planners, it represents a state as a NumPy array `[x, y, theta]` in meters, meters, and radians. The car cannot turn in place or move sideways. The supplied steering function accounts for its turning constraint; you implement neither car dynamics nor a steering solver.

The runner builds a `DubinsWorld`, creates a seeded random generator, and calls `rrt(world, start, goal, rng, step, goal_bias, goal_tol, max_iters)`. The tree is a list of pairs `(state, parent_index)` in insertion order. Its root is `(start, -1)`. For example, `tree[i][0]` is node *i*'s state, and `tree[i][1]` is the index of its predecessor. An index identifies a node in this list; it is not a grid coordinate.

Provided scaffold and interface. The starter supplies the loop, 5% goal-biased sampling, node insertion, collision rejection, goal test, counters, and path recovery. Each iteration first decides whether to sample the goal; otherwise it calls `world.sample_free(rng)` to obtain a free state. Leave these random draws unchanged so runs with the same seed are reproducible. You need these two provided calls:

- `dubins_steer(world, state, target, step)` tries three forward motions of arc length `step`: maximum left steering, straight, and maximum right steering. It discards colliding arcs and returns the feasible endpoint closest to the target's (x, y) position, or `None` if all three motions are blocked. The endpoint need not reach the target. The target's orientation does not affect selection.
- `world.collision_free(state)` reports whether the car's collision footprint at that state is free. The scaffold uses this as an additional endpoint check after steering.

Your implementation: TODO N. For the current `sample`, find the nearest node already in `tree` by scanning `enumerate(tree)`. Compare squared Euclidean distances $(x_i - x_s)^2 + (y_i - y_s)^2$, ignoring orientation. Update `nearest_i` and `nearest_d2` only when the new distance is strictly smaller. Thus the earliest inserted node wins an exact tie. Do not reorder the tree: its parent indices depend on that order. No spatial-search library is needed.

Your implementation: TODO E. Call `dubins_steer` with `world`, the selected node's state `tree[nearest_i][0]`, `sample`, and `step`. Assign the returned endpoint (or `None`) to `new`. This TODO is a single function call. The supplied code then either rejects the extension or inserts `new` with parent `nearest_i`.

Task 3: checking and running your implementation

Output contract and stopping. The scaffold returns an `RRTResult` with four fields: `path`, `tree`, `samples_drawn`, and `nodes_added`. It tests the goal only after adding a new node. Success means that node's (x, y) distance from the goal is strictly less than `goal_tol`; the final orientation is not constrained. The returned path includes the start and the successful endpoint. If the iteration budget is exhausted, `path` is `None`, but the tree and counters are still returned. `samples_drawn` counts every loop iteration, including rejected extensions and goal samples; `nodes_added` counts accepted extensions, excluding the root. A useful check is `len(tree) == nodes_added + 1`.

The runner supplies a motion length of 0.3 m, goal-sampling probability 0.05, goal tolerance 0.5 m, budget 8000 iterations, and random seed 26. These settings are already in `constants.py`; do not change them or hard-code them into your function. On `maze`, the car's goal is the supplied checkpoint (13.5, 7.5) below the map's goal chamber. The chamber entrance does not admit the car with this task's clearance. Use the `goal` argument exactly as supplied, rather than replacing it with the grid planners' goal.

Check and run. After completing both TODOs, run the Task 3 public tests, then:

```
python ps1/code/run_rrt.py --plot-dir ps1/figures
```

For each map, the command reports the number of samples drawn and the number of nodes added by collision-free extensions. It also plots the tree and returned path. All images are saved in `ps1/figures/`: `ps1_rrt_open.png`, `ps1_rrt_maze.png`, and `ps1_rrt.png` for narrow. Only the narrow image is required in your PDF. The optional `-save-artifacts` flag is not needed. The last printed column is *nodes per sample*; the response below asks for *samples per added node*, which you should compute from the two count columns.

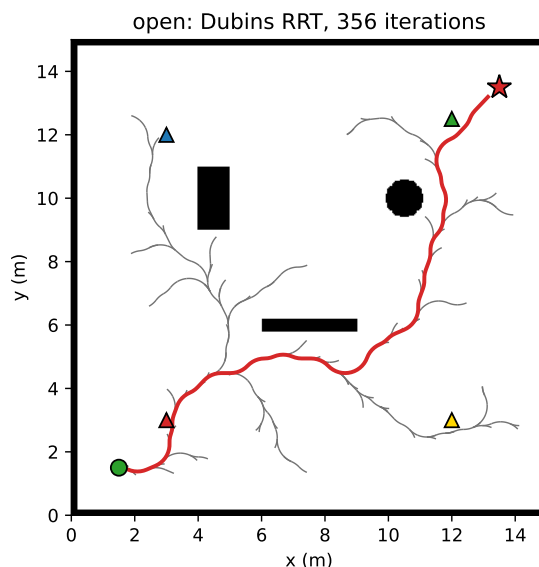


Figure 1: An RRT on the `open` map. Each edge follows a motion primitive that satisfies the car's turning constraint.

Code submission. Submit your implementation in `rrt.py`.

PDF responses.

- (a) Include `ps1_rrt.png`, showing the tree and highlighted path on `narrow`.

Answer / work

- (b) Identify the map with the largest ratio of samples drawn to nodes added and state that ratio. Explain in two or three sentences how the map geometry and motion constraints contribute to it. No separate table is needed.

Answer / work